14/9/26 Exp - 12 (a) Echo client - Server using TCP

Aim:- To implement an Echo client-server application using TCP sockets and verify communication b/w the client and server

Procedure:-

1. Create a TCP echo server using server socket and socket
2. Assign a port number to the server.
3. Compile a Port and execute the TCP Server Program
4. Create a TCP echo client using socket using the server address and port number
5. Connect the client to the server using the server address and Port number
6. Send a message through the client
7. The server receive the message and send it back to the client
8. Display the echoed message at the client
9. Close the to terminate the communication

Result:-

Thus the TCP Echo client - Server application was successfully and was client successfully received the echoed message from the server.

12a

TCP echo server

```
import java.io;
import java.net;
Public class chat server {
    Public static void;
  s = Socket socket()
  s. bind ("local host", 5000))
  s. listen ()
c, addr. s.accept()
  msg = c. recv (1024)
  c. send (msg)
  c. close ()
  c. close ()
```

TCP Echo client

```
  import socket
s = Socket. Socket()
s. connect ("localhost", 5000))
s. send (b" Hellow server")
Print (s. recv (1024), decode())
  s. close.)
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS C:\UMadhan\qaan\Downloads\CN_EXPT_12> java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello
Server: Echo: Hello
I am studying Computer Networks
Server: Echo: I am studying Computer Networks
This is for Testing
Server: Echo: This is for Testing
```

Ex 12 (b) Echo client - Server using

Aim:-

To implement an Echo-client server application using UDP Socket and verify communication b/w client and server

Procedure:-

1. Create a UDP Echo server using datagram socket and datagram Packet
2. Assign a port number to the UDP server.
3. Compile and execute the UDP server Program
4. Create a UDP Echo client using datagram socket and datagram Packet
5. Enter a message through the client
6. The client sends the message as a UDP sends the echoes message at the client
8. Display the sented echoed message at the client
10. Enter bye to terminate the communication

Result:-

Thus the UDP Echo client-server application was successfully implemented, and the client successfully sented echoed message from the system.

128:

```
PS C:\UMadhanraman\Downloads\CN_EXPT_12> java ChatServer
Chat server started on port 5000
Mr. X connected from /127.0.0.1
Mr. Y connected from /127.0.0.1
Mr. X: Hello, How are you?
Mr. Y: I am Fine
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS C:\UMadhanraman\Downloads\CN_EXPT_12> java ChatClient
Enter your name:
Mr: X
Welcome, Mr. X! Type /quit to exit.
Mr. Y has joined the chat.
Hello, How are you?
Mr. Y: I am Fine
```

```
PS C:\UMadhanraman\Downloads\CN_EXPT_12> java ChatClient
Enter your name:
Mr. X has joined the chat.
Mr. Y
Welcome, Mr. Y! Type /quit to exit.
Mr. X: Hello, How are you?
I am Fine
```

UDP Echo Server

```
import socket
s = socket.socket(socket.
    AF_INET,
socket.socket(DGRAM)
s.bind(("localhost", 5001))
msg, addr = s.recvfrom(1024)
s.sendto(msg, addr)
s.close()
```

UDP Echo client

```
import socket
s = socket.socket(socket.
    AF_INET,
socket.socket(DGRAM)
s.sendto(b'Hello',
("localhost", 5001))
msg, addr = s.recvfrom
Print(msg.decode())
s.close()
```

Ex-12 (c) Chat Client - Server Using TCP/IP Sockets

Aim:-
To implement a chat client - server application using sockets and enable communication b/w multiple clients through a server

Procedure:
1. Create a chat server program using socket Programming
2. Start and chat client Program and connect it to the server
3. Run two (or) more client Programs in separate terminals
4. Enter a name for each client
5. Send message from any client to the server
6. The server broadcast the message to other connected clients
7. Continue the communication b/w the clients
9. Enter /quit to terminate a client connection

Result:
Thus the chat client - Server application was successfully implemented and message were smoothly exchanged b/w multiple clients through the server

```
import threading
s = socket.socket()
choice = input("Enter Server (or) Client:")
if choice = "Server":
s.bind(("local host", 5000))
s.listen()
print("Waiting for client....")
conn, address = s.accept()
print("Client connect!")
else: s.connect(("local host", 5000))
    conn = s
def recv():
    while True:
    try: msg = conn.recv(1024).decode()
  if not msg or msg == "exit"
print("Disconnected")
Threading.Thread(target = recv
daemon = True).start()
while True:
    msg = input("you!")
  con.send(msg.encode())
  if msg == 'exit",
break con.out().
```

Output:

Waiting for client

Client connected

you = hello

other = hi.